Analytics Engineering: Session 6

Materializations & Configuration

Agenda

- Core Materializations
- View vs. Table vs. Ephemeral
- Configuring Materializations
- Configuring Grants
- Performance Trade-offs

Core Materializations

How dbt persists your models in the database.

1. **View**: `CREATE VIEW`. Virtual table. No data stored.
2. **Table**: `CREATE TABLE AS SELECT`. Physical storage. Faster to query, slower to build.
3. **Ephemeral**: CTE. Not created in DB. Interpolated into downstream models.
4. **Incremental**: Updates existing table. (Covered in Session 10).

View vs. Table vs. Ephemeral

Type	Pros	Cons	Use Case
View	Fast build, always fresh	Slow query	Staging, lightweight models
Table	Fast query	Slow build, storage cost	BI Marts, heavy logic
Ephemeral	No clutter	Hard to debug	Reusable logic snippets
Incremental	Efficient for large datasets	Complex logic	Large fact tables

Best Practices: Materialization by Layer

- **Staging**: `view`
  - *Reason*: Fast to build, ensures freshness, low storage cost.
- **Intermediate**: `ephemeral` or `view`
  - *Reason*: `ephemeral` keeps warehouse clean (CTEs). `view` is good for debugging.
- **Marts**: `table` or `incremental`
  - *Reason*: Performance. BI tools need fast queries on pre-computed data.

Configuring Materializations

Project-level (in `dbt_project.yml`):

```yaml
models:
  dbt_ie:
    staging:
      +materialized: view
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
```

Model-level (in the SQL file):

```sql
{{ config(materialized='table') }}

select * from {{ ref('stg_customers') }}
```

Configuring Grants

Manage database permissions directly in dbt.

```
models:
  +grants:
    select: ['reporter', 'analyst']

  my_sensitive_model:
    +grants:
      select: ['admin']
```

dbt runs the necessary `GRANT` statements after creating the model.

Performance Trade-offs

- **Start with Views**: They are cheap and easy.
- **Move to Tables**: When query performance becomes an issue for downstream users.
- **Use Ephemeral**: Sparingly, to keep the warehouse clean of intermediate steps.
- **Analyze Build Time**: Check logs after `dbt run` to see which models take longest.

What have we learned in this session

- Configure different materializations for models at project and model levels
- Implement grants for specific roles
- Analyze build logs for different materializations
- Understand performance trade-offs between materialization types

Next Session: Jinja, Macros & Packages.